

SafeStep

Software Setup & Execution Guide

Team: Stack Overlords | HackElite 3.0 | Phase 2 (MVP) Submission

This document explains how to set up and run the SafeStep prototype: the PostgreSQL database, the FastAPI backend, the React (Vite) frontend, and the optional Android app. It mirrors **STARTUP_COMMANDS.md** in the repository, which stays the source of truth if this guide and the code ever drift. Architecture, tech stack, technical challenges, and scope delivered are covered separately in **README.md**.

Project Overview

SafeStep is a women's safety companion built for Sri Lanka — Deter, Detect, Respond, Support — available in Sinhala, Tamil, and English. This submission is the web application, plus an Android build (via Capacitor) of the same frontend with one added native feature.

Core features exercised by this guide:

- **AI Virtual Companion** — fake incoming call ("Theatre Mode") and scheduled safety check-ins
- **Smart Route Intelligence** — crowd-sourced safety reports rendered as a live heatmap
- **Privacy-First Emergency Response** — one-tap SOS with live location and trusted-contact email alerts
- **Multilingual AI Support Chatbot** — Sinhala/Tamil/English, with optional voice input
- **Live location sharing** — revocable, expiring links; passive sharing with trusted contacts
- **Verified Helper Network** — seeded helpers plus live nearby hospitals/police/pharmacies

0. Get the code

Clone the repository (**git clone <https://github.com/Stack-Overlords-Hackelite-3-0/SafeStep.git>**) before proceeding.

```
git clone https://github.com/Stack-Overlords-Hackelite-3-0/SafeStep.git
cd SafeStep
```

1. Prerequisites

- **Python 3.11+** — <https://www.python.org/downloads/>
- **Node.js 18+** (includes npm) — <https://nodejs.org/>
- **PostgreSQL 14+** — install locally, or use Docker Desktop (recommended, see Option A below)
- **Docker Desktop** (recommended) — <https://www.docker.com/products/docker-desktop/>

2. Database — start PostgreSQL

Option A: Docker (recommended)

```
docker compose up -d
```

Starts Postgres on **localhost:5433** (mapped to the container's internal 5432, to avoid clashing with a local Postgres install) with user **safestep**, password **safestep**, database **safestep**. Stop it later with **docker compose down** (add **-v** to also wipe the data volume). If port 5433 is taken, edit the port mapping in **docker-compose.yml** and the port in **backend/.env**'s **DATABASE_URL** to match.

Option B: Local PostgreSQL install

Create the database and user yourself:

```
CREATE USER safestep WITH PASSWORD 'safestep';
CREATE DATABASE safestep OWNER safestep;
```

3. Backend — FastAPI (Python)

```
cd backend

# Windows (PowerShell)
python -m venv venv
venv\Scripts\Activate.ps1

# macOS / Linux
python3 -m venv venv
source venv/bin/activate

pip install -r requirements.txt

# Copy the example env file and adjust if needed
# Windows (PowerShell)
Copy-Item .env.example .env
# macOS / Linux
cp .env.example .env

uvicorn app.main:app --reload --port 8000
```

The backend is now running at **http://localhost:8000** (interactive API docs at **http://localhost:8000/docs**). Database tables are created automatically on startup, and a handful of sample verified helpers are seeded around Colombo the first time it runs.

The AI chatbot works out of the box with a built-in rule-based responder — no API key needed. To wire up a real LLM (Sinhala/Tamil quality is noticeably better), set in **backend/.env**:

```
LLM_PROVIDER=anthropic
ANTHROPIC_API_KEY=sk-ant-...

# or:
LLM_PROVIDER=openai
OPENAI_API_KEY=...
```

4. Frontend — React (Vite)

In a second terminal:

```
cd frontend
npm install

# Windows (PowerShell)
Copy-Item .env.example .env
# macOS / Linux
cp .env.example .env

npm run dev
```

The frontend is now running at **http://localhost:5173**.

Environment variables reference

backend/.env

Variable	Required?	Purpose
DATABASE_URL	Yes	PostgreSQL connection string
JWT_SECRET	Yes	Signs auth tokens; change from the default before any real deployment
JWT_ALGORITHM / JWT_EXPIRE_MINUTES	No	Defaults: HS256 / 1440 (24h)
CORS_ORIGINS	Yes	Comma-separated list of allowed frontend origins
SMTP_HOST / PORT / USER / PASSWORD / MAIL_FROM	No	Outbound email for SOS alerts, invites, password reset. Left blank: emails are skipped
FRONTEND_URL	Yes	Used to build links in emails
LLM_PROVIDER	No	none (default) anthropic openai ollama
ANTHROPIC_API_KEY / OPENAI_API_KEY	No	Required only if LLM_PROVIDER is set to that provider
OLLAMA_BASE_URL / OLLAMA_MODEL	No	Only used when LLM_PROVIDER=ollama
CHAT_ENCRYPTION_KEY	No	Optional dedicated key for chat encryption-at-rest; derived from JWT_SECRET if unset
WHISPER_MODEL_SIZE / DEVICE / COMPUTE_TYPE	No	Tune the local speech-to-text model (chatbot voice input); sensible defaults provided

frontend/.env

Variable	Required?	Purpose
VITE_API_BASE_URL	Yes	Base URL of the backend API. localhost:8000 for web dev; see Section 5 for Android

5. Android app (optional)

The same frontend is also packaged as an Android app via Capacitor (**frontend/android/**), adding one native-only feature: a system-wide floating SOS bubble. This step is not required for local web development — only for building/running the Android app.

Extra prerequisites

- **Android Studio** (includes the Android SDK) — <https://developer.android.com/studio>
- A running backend (step 2) reachable from the phone/emulator

Point the app at a reachable backend

localhost inside an emulator or physical device refers to the device itself, not your machine, so **frontend/.env**'s **VITE_API_BASE_URL** needs to be:

- Android emulator: **http://10.0.2.2:8000** (the emulator's alias for the host machine)
- Physical device: **http://<your machine's LAN IP>:8000** (device and machine on the same network), and **backend/.env**'s **CORS_ORIGINS** must include that origin

```
cd frontend
```

```
# Build the web app and sync it into the Android project
npm run android:sync

# Open the Android project in Android Studio to build/run on a device or emulator
npm run android:open

# Or build + install + launch directly (device/emulator must already be connected)
npm run android:run
```

The native SOS bubble (**android/app/src/main/java/com/safestep/app/SosOverlayPlugin.java**) only activates on Android — it is a no-op when running the same code as a website.

6. Daily workflow (after first-time setup)

```
# Terminal 1 - database (skip if already running)
docker compose up -d

# Terminal 2 - backend
cd backend
venv\Scripts\Activate.ps1 # Windows, or: source venv/bin/activate (macOS/Linux)
uvicorn app.main:app --reload --port 8000

# Terminal 3 - frontend
cd frontend
npm run dev
```

Then open **http://localhost:5173** in your browser.

7. Demo walkthrough (execution steps)

There is no seeded user login — create an account first. Suggested order to exercise every feature end to end:

- **Register** at **/register**, then log in. A JWT is issued and stored client-side; all pages below require this.
- **Dashboard (/)** — passive location sync starts automatically; the floating SOS button (and, on Android, the native SOS bubble) is available from here on.
- **Contacts (/contacts)** — add a trusted contact by email to generate an invite link (**/invite/<token>**); the contact accepts to enable mutual location visibility and SOS email notification.
- **Trigger SOS** — fires **/api/sos/trigger**, emails every trusted contact with an email on file (requires SMTP configured; otherwise it logs and still records the alert), then resolve it to confirm the location fields are wiped.
- **Route Intelligence (/routes)** — submit a safety report (safe/unsafe/well-lit/isolated/harassment/suspicious) and see it appear on the live heatmap.
- **Chatbot (/chatbot)** — ask a safety/legal/helpline question in any of English, Sinhala, or Tamil; optionally try the mic button for voice input (first use downloads the local Whisper model, so it's slower the first time).
- **Helpers (/helpers)** — view the seeded Colombo helper network plus live nearby hospitals/police/pharmacies pulled from OpenStreetMap, sorted by distance, with walking directions.
- **Check-ins (/checkins)** — schedule a safety check-in and try the fake-incoming-call ("Theatre Mode") flow.

- **Share location** — generate a revocable, expiring public share link (`/share/<token>`) and open it in an incognito window to confirm it needs no login.

Full interactive API documentation (every endpoint, request/response schemas) is auto-generated by FastAPI at <http://localhost:8000/docs> once the backend is running.

Troubleshooting

- **"password authentication failed" / can't connect to DB** — make sure **docker compose up -d** is running, or that your local Postgres credentials match **backend/.env**'s **DATABASE_URL**.
- **CORS errors in the browser console** — confirm **frontend/.env**'s **VITE_API_BASE_URL** points at the backend (<http://localhost:8000>) and **backend/.env**'s **CORS_ORIGINS** includes <http://localhost:5173>.
- **Port already in use** — another process is using 8000 or 5173; stop it or change the port (`--port` for uvicorn, **server.port** in **frontend/vite.config.js**).
- **bcrypt / passlib error on install** — make sure **bcrypt==4.0.1** installed correctly (already pinned in **requirements.txt**); reinstall with **pip install -r requirements.txt --force-reinstall** if needed.

Known limitations

- No SMS/offline emergency channel — email is the only out-of-band SOS notification; contacts without an email on file are reported as "unreachable" rather than silently skipped.
- The LLM chatbot upgrade (Anthropic/OpenAI/Ollama) needs an API key/local model to see conversational-quality answers; without one it automatically runs the built-in rule-based knowledge-base responder, which is fully functional but not conversational.
- No Alembic migrations — schema changes ship as additive "ALTER TABLE ... ADD COLUMN IF NOT EXISTS" statements run on backend startup (**backend/app/main.py**).
- The Android build is a local debug build for demo purposes; it hasn't been published to an app store and requires Android Studio/the Android SDK to compile.

Submission info

Team	Stack Overlords
Event	HackElite 3.0 — Phase 2 (MVP)
Repository	See the submitted GitHub link / zip
Demo video (7-10 min)	See README.md and the submission form for the YouTube link
Full technical write-up	README.md — tech stack, architecture, technical challenges, scope delivered